

Coding Assignment 7: Looking Both Ways

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 3, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

7.1 Why this problem

Week 6 built a singly linked list whose every delete paid a walk: to unlink a node you had to find the node whose `next` pointed at it, because the list could only look forward. Week 7 adds one pointer per node — the predecessor link `prev` — and that single change turns “delete a node you already hold a pointer to” from $O(n)$ into $O(1)$. This assignment (practical P7) is that doubly linked list, together with the week’s headline application: sparse polynomials, where only the nonzero terms are stored and two polynomials are added by a single linear merge. You implement both on heap nodes, and every insert or delete must leave both links of the *two-link invariant* in agreement: for every interior node `p`, `p->next->prev == p` and `p->prev->next == p`.

7.2 Your task

Implement the doubly linked list and sparse polynomial ADTs in C. The contract is given to you in `dlist.h` — *do not modify it*. Copy `starter_dlist.c` to `dlist.c` and fill in all fifteen functions.

Part A — the doubly linked list. The node is data plus a forward and a backward link:

```
typedef struct dnode { int data; struct dnode *prev; struct dnode *next; } DNode;

DNode *dlist_insert_front(DNode **head, int x);
DNode *dlist_insert_back(DNode **head, int x);
DNode *dlist_insert_after(DNode *p, int x);
int    dlist_delete_front(DNode **head, int *out);    /* 0 ok / -1 empty */
int    dlist_delete_back(DNode **head, int *out);    /* 0 ok / -1 empty */
int    dlist_delete_node(DNode **head, DNode *target); /* 0 ok / -1 */
DNode *dlist_search(DNode *head, int x);             /* NULL when absent */
int    dlist_print_forward(const DNode *head, int *out, int out_cap);
int    dlist_print_backward(const DNode *head, int *out, int out_cap);
void    dlist_reverse(DNode **head);
void    dlist_free(DNode **head);
```

Insertion is the deck’s two-write pointer surgery, done for *both* links. Inserting `x` after a known node `p` wires the new node between `p` and `p->next`, repairs the old successor’s `prev` *before* overwriting `p->next`, and is $O(1)$ worst case because the pointer `p` is already in hand; `dlist_insert_front`

is the special case at `head`. Because this ADT caches no tail handle, `dlist_insert_back` and `dlist_delete_back` must walk to the last node, so each is $O(n)$ worst case — state that honestly in your docstrings. **Deletion is the whole point of the backward link:** given only a pointer to a node, `dlist_delete_node` rewires `target->prev->next` and `target->next->prev` (moving `head` when the target is first), then frees the node exactly once, in $O(1)$ worst case. `print_forward` and `print_backward` copy the sequence into the caller’s buffer (the “print/display” operation made testable, exactly as Week 5’s `cqueue_display` and Week 6’s `slist_display`); `print_backward` walks to the tail and then back through `prev`. `reverse` flips *both* links of every node.

Part B — sparse polynomials. A polynomial $\sum_i c_i x^{e_i}$ is stored as a chain of `pnode` terms in strictly descending exponent order; a zero coefficient is never stored (the sparsity payoff). The polynomial chain is *singly* linked with a cached `tail` — document why: the merge only ever moves forward, so a `prev` link would buy nothing here, while the cached `tail` makes every `append` $O(1)$.

```
struct pnode { int coeff; int exp; struct pnode *next; };
struct poly { struct pnode *head; struct pnode *tail; };

void poly_init(struct poly *p);
int poly_append_term(struct poly *p, int coeff, int exp); /* 0 ok / -1 fail */
void poly_free(struct poly *p);
int poly_add(const struct poly *a, const struct poly *b, struct poly *out);
```

`poly_add` is a single linear merge of the two descending-exponent chains: copy the larger exponent through, add the coefficients on an equal exponent, suppress a term whose coefficient sums to zero, and when one chain is exhausted append the rest of the other. Build the output in descending order with $O(1)$ tail appends, so the whole addition is $O(n + m)$ worst case for n and m terms. An all-cancelling sum is not an error — it leaves `out` as the zero polynomial (`head == NULL`). `out` must be the zero polynomial on entry and must not alias `a` or `b`; on any failure free partial work and leave `out` empty.

7.3 Constraints

- Representation must be the heap-chained `DNode` (Part A) and `pnode` (Part B) nodes of `dlist.h` — no arrays, no global/static mutable state.
- Every allocation is checked (`malloc` for `NULL`); every node is freed exactly once; no use-after-free, no writes past allocated bounds. Error paths leave the list or polynomial unchanged.
- The two-link invariant must hold after every list operation; the hidden suite re-checks it after each one.
- Correctness is graded against the contract; your code must compile with `gcc -std=c11 -Wall -Wextra -Werror` without errors or warnings.
- Complexity: `insert_front`, `insert_after`, `delete_front`, and `delete_node` are $O(1)$ worst case; `insert_back` and `delete_back` are $O(n)$ worst case (no tail handle); `search` is $O(n)$ worst case and $O(1)$ best case; both prints are $\Theta(n)$; `reverse` is $O(n)$ worst case in $O(1)$ auxiliary space; `poly_append_term` is $O(1)$ worst case; `poly_add` is $O(n + m)$ worst case.

7.4 Worked examples (these are the visible tests)

Example 1 — build the chain and read it both ways. End-insert 14, then 28, then 42. Front-insert 7 (chain $7 \leftrightarrow 14 \leftrightarrow 28 \leftrightarrow 42$). Insert 21 after the node holding 14 (chain $7 \leftrightarrow 14 \leftrightarrow 21 \leftrightarrow 28 \leftrightarrow 42$). `print_forward` writes 7, 14, 21, 28, 42; `print_backward` writes 42, 28, 21, 14, 7. Freeing sets `head` to `NULL`.

Example 2 — delete from both ends and by pointer. From $14 \leftrightarrow 28 \leftrightarrow 42$: `delete_front` returns 14 (chain $28 \leftrightarrow 42$), then `delete_back` returns 42 (chain 28). `search(28)` finds the node; `search(99)` reports absent (`NULL`). Deleting that node by pointer empties the list; `delete_front` on the empty list reports failure.

Example 3 — reverse, and a polynomial addition with cancellation. Reversing $28 \leftrightarrow 35$ gives $35 \leftrightarrow 28$; `print_backward` then reads 28, 35. Reversing again restores $28 \leftrightarrow 35$. Adding $A = 5x^2 + 3$ and $B = -5x^2 + 4x$ gives $4x + 3$: the equal-exponent x^2 terms cancel to coefficient 0 and no zero term is stored, while the unshared $4x$ and the constants copy through.

7.5 What to submit

1. `dlist.c` — your completed implementation (edit `starter_dlist.c`).
2. `dlist.h` — unchanged, but include it so the submission compiles standalone.

7.6 Getting started

1. Copy `starter_dlist.c` to `dlist.c` and fill in the fifteen function bodies.
2. Sanity-check locally:

```
gcc -std=c11 -Wall -Wextra -Werror -o test_visible test_visible.c dlist.c
./test_visible
```

All three worked examples should pass.

3. Submit `dlist.c` and `dlist.h`. The autograder compiles your `dlist.c` against a *hidden* test suite and reports pass/fail. The hidden suite covers: `NULL` head-pointer safety for every operation; front/back/after insert boundaries with neighbour-link repair and returned-node identity; delete of head/middle/tail by pointer using saved node pointers; `delete_node(NULL)` and empty-list refusals with `*out` untouched; `out-is-NULL` calls; `print_forward/print_backward` on empty/single/many with a backward print that is the reverse of the forward print; buffer-cap and `NULL`-buffer refusals with a canary check that nothing is written, plus exact-capacity success and head purity; search identity at head and tail; reverse of empty/single/two/four chains with double-reverse identity; the two-link invariant re-verified after every operation; `INT_MIN/INT_MAX/-1/0` round-trips; polynomial helper boundaries (zero-coefficient skip, descending-order enforcement, double-free safety); `poly_add` argument validation, empty operands, full and partial cancellation, all-zero results, different lengths, and `INT` extremes; and two seeded random campaigns checked operation-by-operation against independent oracles — the list against an int-array model, the polynomial against an exponent-sum model.